Tecnologia em Sistemas para Internet

DISCIPLINA:
ESTRUTURAS DE DADOS

LISTAS DUPLAMENTE LIGADAS

- Prof. Alexandre Ponce de Oliveira

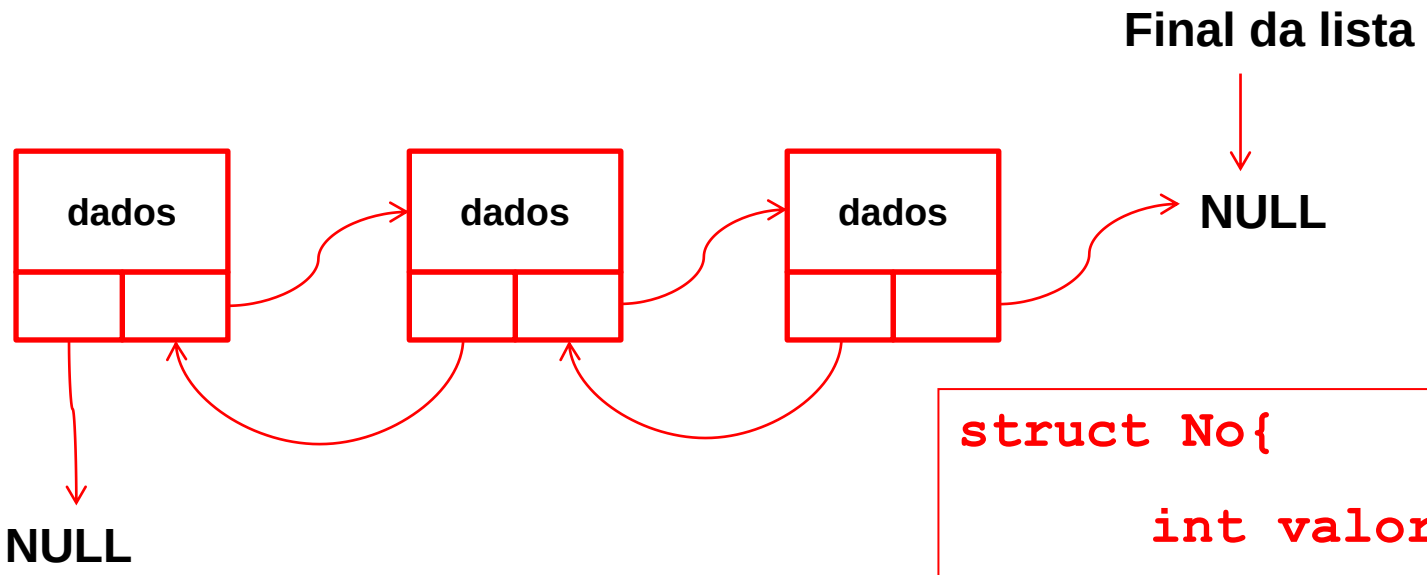
alexandreponceo@gmail.com

Aulas 33 e 34

Lins/SP

Listas Duplamente Ligadas

São representadas por uma estrutura que contém campos de informação e os endereços (ponteiros) do próximo elemento e do elemento anterior:




```
struct No{
        int valor;
        struct No *prox;
        struct No *ant;
};
typedef struct No Lista;
```

Criação da estrutura de uma lista ligada simples

```
#include <stdio.h>
#include <stdlib.h>
struct No {
        int valor;
        struct No *prox;
        struct No *ant;
};
typedef struct No lista;

int main(void)
{       int i;
        lista *cartas = (lista *) malloc(sizeof(lista));
        if(!cartas){
                printf("Impossivel alocar memoria\n");
                exit(1);
        }
        else
            printf("Memoria alocada\n");
        free(cartas);
        return 0;
}
```

Inicialização de uma lista ligada simples

//Função

```
void inicializa (lista *cartas)
{
	cartas->prox = NULL;
	cartas->ant = NULL;
	printf("Lista cartas inicializada\n\n");
}
```

//Código na função main()

```
	inicializa(cartas);
```

Checa se a lista está vazia

//Função

```
int checa_lista(lista *cartas)
{
	if(cartas->prox == NULL)
		return 1;
	else
		return 0;
}
```

//Código na função main()

```
	i = checa_lista(cartas);
	if(i == 1)
	    printf("Lista vazia\n");
	else if (i == 0)
	    printf("Lista não vazia\n");
```

Adiciona um elemento no final da lista

```
//Função
void adiciona_fim(lista *cartas)
{       lista *novo = (lista *) malloc(sizeof(lista));
        if(!cartas)
        {       printf("Impossivel alocar memoria\n");
                exit(1);  }
        printf("Digite uma carta para inserir no fim: ");
        scanf("%d", &novo->valor);
        novo->prox = NULL;
        if(checa_lista(cartas))
        {       cartas->prox = novo;
                novo->ant = cartas;
        } else
        {       lista *aux = cartas->prox;
                while(aux->prox != NULL)
                        aux = aux->prox;
                aux->prox = novo;
                novo->ant = aux;
        }
```

```
//Código na função main()
adiciona_fim(cartas);
```

Adiciona um elemento no início da lista

```
//Função
void adiciona_inicio(lista *cartas)
{
        lista *novo = (lista *) malloc(sizeof(lista));
        if(!novo)
        {       printf("Impossivel alocar memoria\n");
                exit(1);
         }
        printf("Digite uma carta para inserir no inicio: ");
        scanf("%d", &novo->valor);
        lista *aux = cartas->prox;
        cartas->prox = novo;
        novo->prox = aux;
        novo->ant = cartas;
        if (aux != NULL)
          aux->ant = novo;
}

//Código na função main()
        adiciona_inicio(cartas);
```

Mostra os elementos da lista

//Função

```
void mostra_lista(lista *cartas)
{
        if(checa_lista(cartas))
        {
            printf("Lista vazia\n");
               return ;
        }
        lista *aux = cartas->prox;
        while( aux != NULL){
               printf("Cartas = %d\n", aux->valor);
               aux = aux->prox;
        }
}
```

//Código na função main()

```
        mostra_lista(cartas);
```

Mostra os elementos anteriores da lista

//Função

```
void mostra_anterior(lista *cartas)
{
        if(checa_lista(cartas))
        {     printf("Lista vazia\n");
              return ;   }
        lista *aux = cartas;
        printf(“\n\nCartas ");
        while(aux->prox != NULL)
        {       aux = aux->prox;
                printf("-> %d ", aux->valor);   }
        printf(" | ");
        while (aux->ant != NULL)
        {       printf("%d -> ", aux->valor);
                aux = aux->ant;   }
        printf("\n\n\n");
}
```

//Código na função main()

```
mostra_anterior(cartas);
```

Remove um elemento do início da lista

//Função

```
void remove_inicio (lista *cartas)
{
        lista *aux = cartas->prox -> prox;
        lista *head = cartas->prox;
        printf("Removendo o primeiro elemento do baralho \n");
        cartas->prox = cartas->prox -> prox;
        aux->ant = cartas;
        free(head);
}
```

//Código na função main()

```
        remove_inicio(cartas);
```

Remove um elemento do final da lista

```
//Função
void remove_final (lista *cartas)
{
        while (cartas->prox->prox != NULL)
        {
                cartas = cartas->prox ;
        }
        free(cartas->prox);
        cartas->prox = NULL;
        printf("Removendo o ultimo elemento do baralho \n");
}

//Código na função main()
        remove_final(cartas);
```

Limpa a lista

```
//Função
void limpa_lista(lista *cartas)
{       if(!checa_lista(cartas))
        {       lista *aux,*atual,*head;
                head = cartas;
                atual = cartas->prox;
                while(atual->prox != NULL)
                {
                        aux = atual->prox;
                        free(atual);
                        atual = aux;
                }
                if (atual->prox == NULL)
                {    head->prox = NULL;
                   free(atual);
                }
        }
}
                //Código na função main()
                limpa_lista(cartas);
```

Verifica o tamanho da lista

```
//Função
int tamanho_lista (lista *cartas)
{
        int i = 0;
        while (cartas->prox != NULL)
        {
                cartas = cartas->prox;
                i++;
        }
        return i;
}
```

```
//Código na função main()
        i = tamanho_lista(cartas);
        printf("Tamanho da lista = %d\n", i);
```

Exercícios

1) Crie duas funções para adicionar e excluir elementos em uma posição específica da lista duplamente ligada.

2) Crie uma função para adicionar elementos de forma ordenada. Por exemplo, lista = 1, 20 e 38. Com a adição do elemento 28, a lista ficaria da seguinte forma: 1, 20, 28 e 38.

3) Crie um programa com menu de opções com todas as funções implementadas até o momento com listas duplamente ligadas.